Project Concierge — Technical Report Project: Concierge AI Assignment: EduRankAI Intern Challenge — 3-Day Build Status: Working prototype 1. Problem as I understood it The objective was to build a customer-support assistant that answers questions using company knowledge, provides supporting sources, interacts with a mock order service, and escalates questions when it cannot answer reliably. The main engineering challenge is not simply generating a response. It is deciding whether the available evidence supports that response. I selected Project Concierge because it combines information retrieval, language models, API integration, failure handling, and human escalation in one practical system. Assumptions and scope • The supplied sample PDF and refund-policy text file represent the available company knowledge used in the prototype. • The order service is simulated using local mock order data. • The system is a demonstration prototype, not a production deployment. • A similarity threshold of 0.45 is used provisionally to filter retrieved passages. 2. Approach and key decisions Document ingestion and retrieval I implemented PDF and TXT loading using PyMuPDF and Python file handling. Extracted text is split into sentence-aware chunks. The all-MiniLM-L6-v2 Sentence Transformer generates embeddings. These embeddings are normalized and stored in a FAISS IndexFlatIP index. Normalized inner-product search is used to rank semantically similar passages. The index and chunk metadata are saved to disk so the application can load them without rebuilding the embeddings on every startup. Answer generation The RAG service retrieves the top three passages, filters them using the provisional 0.45 similarity threshold, and sends the remaining context to Google Gemini. The prompt instructs the model to use only the supplied context, avoid inventing policies, and acknowledge missing information. Responses include source metadata. If no relevant passages are retrieved, the system refuses to answer and creates a human-support ticket. A phrase-based refusal detector also creates tickets when the model explicitly indicates that it lacks sufficient information. Order service and escalation I implemented a mock order service with three sample orders. It supports simulated slow responses, service errors, and malformed responses. The FastAPI route validates returned data and responds with appropriate HTTP errors. Human escalation tickets include a unique identifier, customer question, reason, status, and timestamp. Tickets are persisted in a JSON file and survive application restarts. Frontend A browser-based dashboard was added using HTML, CSS, and JavaScript. It provides an AI chat interface, order lookup, and escalation-ticket listing. Alternatives and trade-offs • FAISS was selected for straightforward local vector search rather than introducing a hosted vector database. • JSON ticket storage was selected for simplicity rather than adding a database during the prototype. • A fixed similarity threshold was used as an initial baseline; it has not been optimized. • A lightweight HTML/CSS/JavaScript frontend was used instead of introducing a larger frontend framework. 3. Results, with honest uncertainty Offline retrieval evaluation The evaluation dataset contains 40 questions. The retrieval evaluator checks whether at least one of the top three retrieved passages has a similarity score of 0.45 or higher. Metric Result Questions 40 True positives 13 False positives 5 True negatives 22 False negatives 0 Accuracy

87.50% Precision 72.22% Recall 100.00% The system retrieved a qualifying passage for all 13 questions labelled as answerable. Five questions labelled for escalation also retrieved qualifying passages. These five cases show an important limitation: semantic similarity does not prove that a passage supports the customer's claim. For example, a question about a 90-day refund can correctly retrieve the seven-day refund policy, but the final answer must still reject the unsupported 90-day claim. These figures measure retrieval decisions only. They do not establish final answer accuracy, citation faithfulness, or end-to-end safety. Automated testing Eight Pytest tests passed. They cover valid order lookup, case-insensitive order IDs, unknown orders, simulated service errors, malformed order responses, ticket creation, queue retrieval, and unique ticket IDs. The API and frontend were also manually exercised during development. No statistically meaningful production performance or latency claim is made. 4. What failed and what I learned The initial retrieval evaluation exposed five false positives. The cause was semantic overlap between unsupported customer claims and relevant policy passages. The Gemini API also reached its free-tier request quota during testing. This prevented repeated end-to-end model evaluation. I preserved offline retrieval evaluation and service testing so development could continue without further model calls. The primary lessons were: 1. Retrieved content can be relevant without supporting the requested claim. 2. Retrieval quality and answer correctness require separate evaluation. 3. External model services need explicit quota and availability handling. 4. Failure cases should be documented rather than hidden. 5. A working prototype is not equivalent to a production-ready system. Detailed failure observations are recorded in FAILURES.md. 5. Risks, limitations, and two-week improvement plan The current system has several limitations: • The evaluation set needs verified expected answers and source passages for every question. • Ambiguous questions and questions requiring evidence from multiple documents need stronger coverage. • Contradictions, outdated policies, and document version changes have not been comprehensively evaluated. • The refusal detector uses phrase matching and can miss unsupported answers expressed in other wording. • The order lookup endpoint is not yet integrated into the conversational RAG workflow. • The simulated slow service does not yet demonstrate a full timeout and retry policy. • JSON ticket persistence is unsuitable for concurrent production traffic. • Final answer correctness, citation accuracy, latency, and cost have not been measured across the full evaluation set. With two additional weeks, I would prioritize: 1. Expand and annotate the evaluation set with correct answers and source passages. 2. Add answer-grounding and citation-correctness evaluation. 3. Test at least ten adversarial inputs and document their outcomes. 4. Add document versioning and incremental index updates. 5. Integrate order lookup into the assistant with appropriate validation and authorization. 6. Add timeouts, retries, and resilience checks for service failures. 7. Replace JSON ticket storage with PostgreSQL. 8. Compare two meaningful retrieval or model configurations on the same dataset. 9. Measure latency, token usage, and API cost. 10. Add automated regression evaluation for future changes. Launch recommendation: Suitable for a controlled prototype demonstration, not for

unrestricted customer-facing production use. Human review, policy verification, monitoring, and stronger evaluation are required before launch. 6. How to run everything Requirements: Python 3.12, Git, and a Google Gemini API key. git clone https://github.com/chidusurabhi123-collab/concierge-ai cd concierge-ai

python -m venv .venv source .venv/bin/activate

python -m pip install -r requirements.txt Create .env in the project root: GEMINI_API_KEY=gemini_api_key Start the application: uvicorn app.main:app –reload Open the dashboard at http://127.0.0.1:8000/ and API documentation at http://127.0.0.1:8000/docs. Run automated tests: python -m pytest tests/ -v Run offline retrieval evaluation: python -m evaluation.evaluate_retrieval AI assistance disclosure: AI coding assistance was used for implementation guidance, debugging, code drafting, and documentation. I am responsible for reviewing and explaining the submitted implementation.